

SKILL 2:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill2.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill2.c

/* Skill 2: Main loop, prompt, read input, handle exit, interactive loop */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char *line = NULL;
    size_t cap = 0;
    ssize_t len;

    while (1) {
        printf("wsh> ");
        fflush(stdout);

        len = getline(&line, &cap, stdin);
        if (len == -1) { /* EOF (Ctrl+D) */
            printf("\nextit\n");
            break;
        }
        if (len > 0 && line[len - 1] == '\n')
            line[len - 1] = '\0';

        if (strcmp(line, "exit") == 0) {
            printf("exit\n");
            break;
        }
        if (strlen(line) == 0)
            continue;

        printf("you typed: %s\n", line);
    }

    free(line);
    return 0;
}

mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill2.c -o skill2
mahij@Mahija:~$ ./skill2
wsh> you typed: hello world
wsh> exit
mahij@Mahija:~$ |
```

SKILL 3:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill3.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill3.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BUF_INIT 8

int main(void) {
    int c;
    size_t cap = BUF_INIT, len = 0;
    char *buf = malloc(cap);

    printf("wsh> ");
    fflush(stdout);

    while ((c = getchar()) != EOF) {
        if (c == '\n') {
            buf[len] = '\0';
            printf("\ncommand buffer -> \"%s\" (final length=%zu)\n", buf, len);
            if (strcmp(buf, "exit") == 0) break;
            len = 0;
            printf("wsh> ");
            fflush(stdout);
            continue;
        }
        if (c == 0x08 || c == 0x7f) {
            if (len > 0) {
                len--;
                printf("\b \b");
            }
            continue;
        }
        if (len + 1 >= cap) {
            cap *= 2;
            buf = realloc(buf, cap);
        }
        buf[len++] = (char)c;
    }
    free(buf);
    return 0;
}

mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill3.c -o skill3
mahij@Mahija:~$ ./skill3
wsh>
command buffer -> "helo world" (final length=10)
wsh>
command buffer -> "exit" (final length=4)
mahij@Mahija:~$ |
```

SKILL 4:

mahij@Mahija: ~ × +

mahij@Mahija:~\$ nano skill4.c
mahij@Mahija:~\$ |

GNU nano 8.7.1skill4.c

```
/* Skill 4: Dynamically resizing buffer + simple linked list history */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct node {
    char *cmd;
    struct node *next;
} node_t;

static node_t *head = NULL;

void push_history(const char *cmd) {
    node_t *n = malloc(sizeof(node_t));
    n->cmd = strdup(cmd);
    n->next = head;
    head = n;
}

void free_history(void) {
    node_t *cur = head;
    while (cur) {
        node_t *next = cur->next;
        free(cur->cmd);
        free(cur);
        cur = next;
    }
}

char *read_dynamic_line(void) {
    size_t cap = 4, len = 0;
    char *buf = malloc(cap);
    int c;
    while ((c = getchar()) != EOF && c != '\n') {
        if (len + 1 >= cap) {
            cap *= 2;
            buf = realloc(buf, cap);
            printf("[buffer grown to %zu bytes]\n", cap);
        }
        buf[len++] = (char)c;
    }
    buf[len] = '\0';
    return buf;
}

int main(void) {
    char *line;
    printf("wsh> ");
    while ((line = read_dynamic_line()) && strlen(line) > 0) {
        push_history(line);
        printf("stored: %s\n", line);
        if (strcmp(line, "exit") == 0) { free(line); break; }
        free(line);
        printf("wsh> ");
    }

    printf("\n-- history (most recent first) --\n");
    for (node_t *cur = head; cur; cur = cur->next)
        printf("  %s\n", cur->cmd);

    free_history();
    return 0;
}
```

mahij@Mahija: ~ × +

mahij@Mahija:~\$ gcc skill4.c -o skill4
mahij@Mahija:~\$./skill4
wsh> [buffer grown to 8 bytes]
[buffer grown to 16 bytes]
stored: ls -la /home
wsh> stored: pwd
wsh> [buffer grown to 8 bytes]
stored: exit
-- history (most recent first) --
 exit
 pwd
 ls -la /home
mahij@Mahija:~\$ |

SKILL 5:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill5.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill5.c

/* Skill 5: Tokenizer - split input into tokens on whitespace, debug output */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TOKENS 64

int tokenize(char *line, char **tokens) {
    int n = 0;
    char *tok = strtok(line, " \t");
    while (tok != NULL && n < MAX_TOKENS - 1) {
        tokens[n++] = tok;
        tok = strtok(NULL, " \t");
    }
    tokens[n] = NULL;
    return n;
}

int main(void) {
    char line[256];
    char *tokens[MAX_TOKENS];

    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        char copy[256];
        strcpy(copy, line);
        int n = tokenize(copy, tokens);

        printf("input: \"%s\"\n", line);
        printf("token count: %d\n", n);
        for (int i = 0; i < n; i++)
            printf(" token[%d] = \"%s\"\n", i, tokens[i]);
    }
    return 0;
}

mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill5.c -o skill5
mahij@Mahija:~$ ./skill5
input: "ls -la /home/praks"
token count: 3
token[0] = "ls"
token[1] = "-la"
token[2] = "/home/praks"
input: "cat process.c"
token count: 2
token[0] = "cat"
token[1] = "process.c"
mahij@Mahija:~$ |
```

SKILL 6:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill6.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill6.c

/* Skill 6: Simple parser - build a parse structure from tokens, detect errors */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *argv[64];
    int argc;
    int valid;
} parsed_cmd_t;

parsed_cmd_t parse(char *line) {
    parsed_cmd_t pc = { .argc = 0, .valid = 1 };
    char *tok = strtok(line, " \t");
    while (tok) {
        pc.argv[pc.argc++] = tok;
        tok = strtok(NULL, " \t");
    }
    pc.argv[pc.argc] = NULL;
    if (pc.argc == 0) pc.valid = 0; /* empty command -> invalid */
    return pc;
}

int main(void) {
    char line[256];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strcmp(line, "exit") == 0) break;

        char copy[256];
        strcpy(copy, line);
        parsed_cmd_t pc = parse(copy);

        if (!pc.valid) {
            printf("parse error: empty command\n");
            continue;
        }
        printf("parsed OK -> command=\"%s\", argc=%d\n", pc.argv[0], pc.argc);
        for (int i = 1; i < pc.argc; i++)
            printf("  arg[%d] = %s\n", i, pc.argv[i]);
    }
    return 0;
}

mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill6.c -o skill6
mahij@Mahija:~$ ./skill6
parsed OK -> command="gcc", argc=4
  arg[1] = process.c
  arg[2] = -o
  arg[3] = process
parse error: empty command
mahij@Mahija:~$ |
```

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill7.c
mahij@Mahija:~$ |
```

```
GNU nano 8.7.1 skill7.c

/* Skill 7: Single quotes (literal, no expansion) vs Double quotes (var expansion) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

void expand_var(const char *name, char *out) {
    char *val = getenv(name);
    strcpy(out, val ? val : "");
}

/* returns a newly-alloc'd, processed token */
char *process_token(const char *tok) {
    char result[512] = {0};
    int ri = 0;
    size_t i = 0, n = strlen(tok);

    if (n >= 2 && tok[0] == '\'' && tok[n - 1] == '\'' ) {
        /* single quotes: copy literally, no expansion */
        strncpy(result, tok + 1, n - 2);
        return strdup(result);
    }
    if (n >= 2 && tok[0] == '"' && tok[n - 1] == '"' ) {
        i = 1;
        while (i < n - 1) {
            if (tok[i] == '$') {
                char name[128] = {0};
                int ni = 0;
                i++;
                while (i < n - 1 && (isalnum(tok[i]) || tok[i] == '_'))
                    name[ni++] = tok[i++];
                char val[256];
                expand_var(name, val);
                strcpy(result + ri, val);
                ri += strlen(val);
            } else {
                result[ri++] = tok[i++];
            }
        }
        return strdup(result);
    }
    return strdup(tok);
}

int main(void) {
    char line[512];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strcmp(line, "exit") == 0) break;
        char *out = process_token(line);
        printf("raw: %s\n", line);
        printf("out: %s\n", out);
        free(out);
    }
    return 0;
}
```

```
> mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill7.c -o skill7
mahij@Mahija:~$ ./skill7
raw: echo 'hello $USER'
out: echo 'hello $USER'
raw: echo "hello $USER"
out: echo "hello $USER"
mahij@Mahija:~$ |
```

SKILL 8:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill18.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill18.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *unescape(const char *tok) {
    char result[512];
    int ri = 0;
    for (size_t i = 0; tok[i]; i++) {
        if (tok[i] == '\\' && tok[i + 1] != '\0') {
            result[ri++] = tok[++i];
        } else {
            result[ri++] = tok[i];
        }
    }
    result[ri] = '\0';
    return strdup(result);
}

int main(void) {
    char line[512];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strcmp(line, "exit") == 0) break;
        char *out = unescape(line);
        printf("raw: %s\n", line);
        printf("out: %s\n", out);
        free(out);
    }
    return 0;
}

mahij@Mahija:~$ gcc skill18.c -o skill18
mahij@Mahija:~$ ./skill18
raw: echo hello\ world
out: echo hello world
raw: ls\ -la
out: ls -la
mahij@Mahija:~$ |
```

SKILL 9:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill9.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill9.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[256];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strcmp(line, "exit") == 0) break;
        if (strlen(line) == 0) continue;

        char *argv[64];
        int n = 0;
        char *tok = strtok(line, " ");
        while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
        argv[n] = NULL;

        pid_t pid = fork();
        if (pid == 0) {
            execvp(argv[0], argv);
            fprintf(stderr, "wsh: command not found: %s\n", argv[0]);
            _exit(127);
        } else if (pid > 0) {
            int status;
            waitpid(pid, &status, 0);
            printf("[child pid=%d exited with status %d]\n", pid, WEXITSTATUS(status));
        } else {
            perror("fork");
        }
    }
    return 0;
}

mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill9.c -o skill9
mahij@Mahija:~$ ./skill9
hi
[child pid=432 exited with status 0]
wsh: command not found: nosuchcmd
[child pid=433 exited with status 127]
mahij@Mahija:~$ |
```

Skill 10:

```
mahij@Mahija: ~ × +
mahij@Mahija:~$ nano skill10.c
mahij@Mahija:~$ |
```

```
GNU nano 8.7.1 skill10.c

/* Skill 10: PATH resolution - manually search PATH dirs for an executable */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

char *which(const char *cmd) {
    if (strchr(cmd, '/')) { /* already a path */
        if (access(cmd, X_OK) == 0) return strdup(cmd);
        return NULL;
    }
    char *path = getenv("PATH");
    if (!path) return NULL;
    char *pathcopy = strdup(path);
    char *dir = strtok(pathcopy, ":");
    static char full[1024];
    while (dir) {
        snprintf(full, sizeof(full), "%s/%s", dir, cmd);
        if (access(full, X_OK) == 0) {
            free(pathcopy);
            return strdup(full);
        }
        dir = strtok(NULL, ":");
    }
    free(pathcopy);
    return NULL;
}

int main(void) {
    char line[256];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strcmp(line, "exit") == 0) break;
        if (strlen(line) == 0) continue;

        char *resolved = which(line);
        if (resolved) {
            printf("%s -> %s\n", line, resolved);
            free(resolved);
        } else {
            printf("%s: command not found in PATH\n", line);
        }
    }
    return 0;
}
```

```
mahij@Mahija: ~ × +
mahij@Mahija:~$ gcc skill10.c -o skill10
mahij@Mahija:~$ ./skill10
ls -> /usr/bin/ls
gcc -> /usr/bin/gcc
nosuchcmd: command not found in PATH
mahij@Mahija:~$ |
```

Skill 11:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill11.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill11.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

char *expand(const char *line) {
    char result[1024];
    int ri = 0;
    for (size_t i = 0; line[i]; ) {
        if (line[i] == '$' && (isalpha(line[i+1]) || line[i+1] == '_')) {
            char name[128]; int ni = 0;
            i++;
            while (isalnum(line[i]) || line[i] == '_') name[ni++] = line[i++];
            name[ni] = '\0';
            char *val = getenv(name);
            if (!val) { printf("[note] %s is undefined -> expands to empty\n", name); val = ""; }
            strcpy(result + ri, val);
            ri += strlen(val);
        } else {
            result[ri++] = line[i++];
        }
    }
    result[ri] = '\0';
    return strdup(result);
}

int main(void) {
    char line[1024];
    while (fgets(line, sizeof(line), stdin)) {
        line[strlen(line)] = '\0';
        if (strcmp(line, "exit") == 0) break;
        char *out = expand(line);
        printf("raw: %s\n", line);
        printf("out: %s\n", out);
        free(out);
    }
    return 0;
}
```

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill11.c -o skill11
mahij@Mahija:~$ ./skill11
[note] $USER is undefined -> expands to empty
raw: echo $USER
out: echo
[note] $UNDEFINED_VAR is undefined -> expands to empty
raw: echo $UNDEFINED_VAR
out: echo
mahij@Mahija:~$ |
```

Skill 12:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill12.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill12.c

/* Skill 12: Built-in dispatch table - cd, pwd, exit, export as in-process commands */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

typedef int (*builtin_fn)(char **argv);

int b_cd(char **argv) {
    if (!argv[1]) { fprintf(stderr, "cd: missing argument\n"); return 1; }
    if (chdir(argv[1]) != 0) { perror("cd"); return 1; }
    return 0;
}

int b_pwd(char **argv) {
    (void)argv;
    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd))) printf("%s\n", cwd);
    return 0;
}

int b_export(char **argv) {
    if (!argv[1]) { fprintf(stderr, "export: missing NAME=VALUE\n"); return 1; }
    char *eq = strchr(argv[1], '=');
    if (!eq) { fprintf(stderr, "export: invalid syntax\n"); return 1; }
    *eq = '\0';
    setenv(argv[1], eq + 1, 1);
    printf("exported %s=%s\n", argv[1], eq + 1);
    return 0;
}

int b_exit(char **argv) { (void)argv; exit(0); }

typedef struct { const char *name; builtin_fn fn; } dispatch_t;
dispatch_t table[] = {
    {"cd", b_cd}, {"pwd", b_pwd}, {"export", b_export}, {"exit", b_exit}, {NULL, NULL}
};

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[512];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;

        char *argv[64]; int n = 0;
        char *tok = strtok(line, " ");
        while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
        argv[n] = NULL;

        int matched = 0;
        for (int i = 0; table[i].name; i++) {
            if (strcmp(argv[0], table[i].name) == 0) {
                matched = 1;
                table[i].fn(argv);
                break;
            }
        }
        if (!matched) printf("wsh: '%s' is not a builtin\n", argv[0]);
    }
    return 0;
}

mahij@Mahija:~ × +

mahij@Mahija:~$ gcc skill12.c -o skill12
mahij@Mahija:~$ ./skill12
/home/claude/oss/src
/tmp
exported COURSE=OSSP
wsh: 'ls' is not a builtin
mahij@Mahija:~$ |
```

Skill 13:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill13.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill13.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define HIST_CAP 100
char *history[HIST_CAP];
int hist_count = 0;

void add_history(const char *cmd) {
    if (hist_count >= HIST_CAP) {
        free(history[0]);
        memmove(history, history + 1, sizeof(char *) * (HIST_CAP - 1));
        hist_count--;
    }
    history[hist_count++] = strdup(cmd);
}

void print_history(void) {
    for (int i = 0; i < hist_count; i++)
        printf(" %d %s\n", i + 1, history[i]);
}

int main(void) {
    char line[256];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        add_history(line);
        if (strcmp(line, "history") == 0) {
            print_history();
        } else {
            printf("ran: %s\n", line);
        }
    }
    for (int i = 0; i < hist_count; i++) free(history[i]);
    return 0;
}
```

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill13.c -o skill13
mahij@Mahija:~$ ./skill13
ran: ls -la
ran: pwd
1.  ls -la
2.  pwd
3.  history
mahij@Mahija:~$ |
```

Skill 14:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill14.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill14.c

/* Skill 14: Pipes - support "cmd1 | cmd2" using pipe()+dup2()+fork() */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

void run_simple(char **argv) {
    execvp(argv[0], argv);
    fprintf(stderr, "wsh: command not found: %s\n", argv[0]);
    _exit(127);
}

char **split(char *s) {
    char **argv = malloc(sizeof(char *) * 64);
    int n = 0;
    char *tok = strtok(s, " ");
    while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
    argv[n] = NULL;
    return argv;
}

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[512];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        char *pipe_pos = strchr(line, '|');
        if (!pipe_pos) {
            pid_t pid = fork();
            if (pid == 0) run_simple(split(line));
            waitpid(pid, NULL, 0);
            continue;
        }

        *pipe_pos = '\0';
        char *left = line, *right = pipe_pos + 1;
        int fd[2];
        pipe(fd);

        pid_t p1 = fork();
        if (p1 == 0) {
            dup2(fd[1], STDOUT_FILENO);
            close(fd[0]); close(fd[1]);
            run_simple(split(left));
        }
        pid_t p2 = fork();
        if (p2 == 0) {
            dup2(fd[0], STDIN_FILENO);
            close(fd[0]); close(fd[1]);
            run_simple(split(right));
        }
        close(fd[0]); close(fd[1]);
        waitpid(p1, NULL, 0);
        waitpid(p2, NULL, 0);
        printf("[pipeline complete]\n");
    }
    return 0;
}

mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill14.c -o skill14
mahij@Mahija:~$ ./skill14
42
[pipeline complete]
pid_t pid = fork();
if (pid == 0) {
    printf("Child PID : %d\n", getpid());
    printf("Parent PID : %d\n", getppid());
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);
}
[pipeline complete]
mahij@Mahija:~$ |
```

Skill 15:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill15.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill15.c

/* Skill 15: Multiple pipes - arbitrary-length pipelines, descriptor array mgmt */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAXCMDS 16

char **split(char *s) {
    char **argv = malloc(sizeof(char *) * 64);
    int n = 0;
    char *tok = strtok(s, " ");
    while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
    argv[n] = NULL;
    return argv;
}

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[512];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        char *cmds[MAXCMDS];
        int ncmds = 0;
        char *tok = strtok(line, "|");
        while (tok) { cmds[ncmds++] = tok; tok = strtok(NULL, "|"); }

        int npipes = ncmds - 1;
        int (*pipes)[2] = malloc(sizeof(int[2]) * (npipes > 0 ? npipes : 1));
        for (int i = 0; i < npipes; i++) pipe(pipes[i]);

        pid_t pids[MAXCMDS];
        for (int i = 0; i < ncmds; i++) {
            pids[i] = fork();
            if (pids[i] == 0) {
                if (i > 0) dup2(pipes[i - 1][0], STDIN_FILENO);
                if (i < ncmds - 1) dup2(pipes[i][1], STDOUT_FILENO);
                for (int j = 0; j < npipes; j++) { close(pipes[j][0]); close(pipes[j][1]); }
                execvp(split(cmds[i])[0], split(cmds[i]));
                fprintf(stderr, "wsh: command not found\n");
                _exit(127);
            }
        }
        for (int j = 0; j < npipes; j++) { close(pipes[j][0]); close(pipes[j][1]); }
        for (int i = 0; i < ncmds; i++) waitpid(pids[i], NULL, 0);
        printf("[%d-stage pipeline complete]\n", ncmds);
        free(pipes);
    }
    return 0;
}

mahij@Mahija:~$ gcc skill15.c -o skill15
mahij@Mahija:~$ ./skill15
6
[3-stage pipeline complete]
mahij@Mahija:~$ |
```

Skill 16:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill16.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill16.c

/* Skill 16: Input (<) and output (>) redirection */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[512];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        char *infile = NULL, *outfile = NULL;
        char *in_pos = strchr(line, '<');
        char *out_pos = strchr(line, '>');
        if (in_pos) { *in_pos = '\0'; infile = strtok(in_pos + 1, " "); }
        if (out_pos) { *out_pos = '\0'; outfile = strtok(out_pos + 1, " "); }

        char *argv[64]; int n = 0;
        char *tok = strtok(line, " ");
        while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
        argv[n] = NULL;

        pid_t pid = fork();
        if (pid == 0) {
            if (infile) {
                int fd = open(infile, O_RDONLY);
                if (fd < 0) { perror("open (input)"); _exit(1); }
                dup2(fd, STDIN_FILENO); close(fd);
            }
            if (outfile) {
                int fd = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
                dup2(fd, STDOUT_FILENO); close(fd);
            }
            execvp(argv[0], argv);
            fprintf(stderr, "wsh: command not found: %s\n", argv[0]);
            _exit(127);
        }
        waitpid(pid, NULL, 0);
        printf("[done]\n");
    }
    return 0;
}

> mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill16.c -o skill16
mahij@Mahija:~$ ./skill16
3
[done]
[done]
mahij@Mahija:~$ |
```

Skill 17:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill17.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill17.c

/* Skill 17: Append (>>) and stderr (2>) redirection */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[512];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        char *appendfile = NULL, *errfile = NULL;
        char *app_pos = strstr(line, ">>");
        char *err_pos = strstr(line, "2>");
        if (app_pos) { *app_pos = '\0'; appendfile = strtok(app_pos + 2, " "); }
        if (err_pos) { *err_pos = '\0'; errfile = strtok(err_pos + 2, " "); }

        char *argv[64]; int n = 0;
        char *tok = strtok(line, " ");
        while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
        argv[n] = NULL;

        pid_t pid = fork();
        if (pid == 0) {
            if (appendfile) {
                int fd = open(appendfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
                dup2(fd, STDOUT_FILENO); close(fd);
            }
            if (errfile) {
                int fd = open(errfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
                dup2(fd, STDERR_FILENO); close(fd);
            }
            execvp(argv[0], argv);
            fprintf(stderr, "wsh: command not found: %s\n", argv[0]);
            _exit(127);
        }
        waitpid(pid, NULL, 0);
        printf("[done]\n");
    }
    return 0;
}

> mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill17.c -o skill17
mahij@Mahija:~$ ./skill17

[done]
[done]
mahij@Mahija:~$ |
```

Skill 18:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill18.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill18.c

/* Skill 18: Combined redirection 2>&1 - duplicate fds, merge streams */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[512];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        int merge = 0;
        char *merge_pos = strstr(line, "2>&1");
        if (merge_pos) { merge = 1; *merge_pos = '\0'; }

        char *outfile = NULL;
        char *out_pos = strchr(line, '>');
        if (out_pos) { *out_pos = '\0'; outfile = strtok(out_pos + 1, " "); }

        char *argv[64]; int n = 0;
        char *tok = strtok(line, " ");
        while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
        argv[n] = NULL;

        pid_t pid = fork();
        if (pid == 0) {
            if (outfile) {
                int fd = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
                dup2(fd, STDOUT_FILENO); close(fd);
            }
            if (merge) dup2(STDOUT_FILENO, STDERR_FILENO);
            execvp(argv[0], argv);
            fprintf(stderr, "wsh: command not found: %s\n", argv[0]);
            _exit(127);
        }
        waitpid(pid, NULL, 0);
        printf("[done, merge=%d]\n", merge);
    }
    return 0;
}

mahij@Mahija:~ × +

mahij@Mahija:~$ gcc skill18.c -o skill18
mahij@Mahija:~$ ./skill18
[done, merge=1]
mahij@Mahija:~$ |
```

Skill 19:

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ nano skill19.c
mahij@Mahija:~$ |

GNU nano 8.7.1 skill19.c

/* Skill 19: Background jobs with '&' - non-blocking launch, job table entry */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

typedef struct { int id; pid_t pid; char cmd[128]; char state[16]; } job_t;
job_t jobs[64];
int njobs = 0;

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[256];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        int bg = 0;
        size_t len = strlen(line);
        if (len > 0 && line[len - 1] == '&') { bg = 1; line[len - 1] = '\0'; }

        char *argv[64]; int n = 0;
        char linecopy[256]; strcpy(linecopy, line);
        char *tok = strtok(linecopy, " ");
        while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
        argv[n] = NULL;

        pid_t pid = fork();
        if (pid == 0) {
            execvp(argv[0], argv);
            fprintf(stderr, "wsh: command not found: %s\n", argv[0]);
            _exit(127);
        }
        if (bg) {
            job_t *j = &jobs[njobs++];
            j->id = njobs; j->pid = pid;
            strncpy(j->cmd, line, sizeof(j->cmd));
            strcpy(j->state, "Running");
            printf("[%d] %d %s\n", j->id, pid, line);
        } else {
            waitpid(pid, NULL, 0);
            printf("[done]\n");
        }
    }
    for (int i = 0; i < njobs; i++) waitpid(jobs[i].pid, NULL, 0);
    return 0;
}

mahij@Mahija: ~ × +

mahij@Mahija:~$ gcc skill19.c -o skill19
mahij@Mahija:~$ ./skill19
[1] 447 sleep 1
hi
[done]
mahij@Mahija:~$ |
```

mahij@Mahija: ~ × +

mahij@Mahija:~\$ nano skill20.c
mahij@Mahija:~\$ |

GNU nano 8.7.1skill20.c

```
/* Skill 20: "jobs" listing + "fg" foreground switching */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

typedef struct { int id; pid_t pid; char cmd[128]; char state[16]; } job_t;
job_t jobs[64];
int njobs = 0;

void list_jobs(void) {
    for (int i = 0; i < njobs; i++)
        printf("[%d] %d %-8s %s\n", jobs[i].id, jobs[i].pid, jobs[i].state, jobs[i].cmd);
}

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    char line[256];
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        if (strcmp(line, "jobs") == 0) { list_jobs(); continue; }
        if (strncmp(line, "fg ", 3) == 0) {
            int id = atoi(line + 3);
            for (int i = 0; i < njobs; i++) {
                if (jobs[i].id == id) {
                    printf("bringing [%d] pid=%d (%s) to foreground\n", id, jobs[i].pid, jobs[i].cmd);
                    int status;
                    waitpid(jobs[i].pid, &status, 0);
                    strcpy(jobs[i].state, "Done");
                    printf("[%d] Done %s\n", id, jobs[i].cmd);
                }
            }
            continue;
        }

        int bg = 0;
        size_t len = strlen(line);
        if (len > 0 && line[len - 1] == '&') { bg = 1; line[len - 1] = '\0'; }

        char *argv[64]; int n = 0;
        char linecopy[256]; strcpy(linecopy, line);
        char *tok = strtok(linecopy, " ");
        while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
        argv[n] = NULL;

        pid_t pid = fork();
        if (pid == 0) { execvp(argv[0], argv); _exit(127); }

        if (bg) {
            job_t *j = &jobs[njobs++];
            j->id = njobs; j->pid = pid;
            strncpy(j->cmd, line, sizeof(j->cmd));
            strcpy(j->state, "Running");
            printf("[%d] %d %s\n", j->id, pid, line);
        } else {
            waitpid(pid, NULL, 0);
        }
    }
    return 0;
}
```

mahij@Mahija: ~ × +

mahij@Mahija:~\$ gcc skill20.c -o skill20
mahij@Mahija:~\$./skill20
[1] 451 sleep 1
[1] 451 Running sleep 1
bringing [1] pid=451 (sleep 1) to foreground
[1] Done sleep 1
mahij@Mahija:~\$ |

Skill 21:

mahij@Mahija: ~ × +

mahij@Mahija:~\$ nano skill21.c
mahij@Mahija:~\$ |

GNU nano 8.7.1skill21.c

```
/* Skill 21: Signal handling - SIGINT/SIGTSTP, process groups, forward to foreground job */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

volatile pid_t fg_pid = 0;

void sigint_handler(int sig) {
    (void)sig;
    if (fg_pid > 0) {
        printf("\n[wsh] forwarding SIGINT to foreground pid %d\n", fg_pid);
        kill(fg_pid, SIGINT);
    } else {
        printf("\n[wsh] SIGINT caught, shell continues (no foreground job)\n");
    }
}

void sigtstp_handler(int sig) {
    (void)sig;
    if (fg_pid > 0) {
        printf("\n[wsh] forwarding SIGTSTP to foreground pid %d (suspend)\n", fg_pid);
        kill(fg_pid, SIGTSTP);
    }
}

int main(void) {
    setvbuf(stdout, NULL, _IOLBF, 0);
    struct sigaction sa_int = { .sa_handler = sigint_handler };
    struct sigaction sa_tstp = { .sa_handler = sigtstp_handler };
    sigaction(SIGINT, &sa_int, NULL);
    sigaction(SIGTSTP, &sa_tstp, NULL);

    char line[256];
    printf("[wsh] signal handlers registered for SIGINT and SIGTSTP\n");
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;
        if (strcmp(line, "exit") == 0) break;

        char *argv[64]; int n = 0;
        char linecopy[256]; strcpy(linecopy, line);
        char *tok = strtok(linecopy, " ");
        while (tok) { argv[n++] = tok; tok = strtok(NULL, " "); }
        argv[n] = NULL;

        pid_t pid = fork();
        if (pid == 0) {
            setpgid(0, 0);
            execvp(argv[0], argv);
            _exit(127);
        }
        setpgid(pid, pid);
        fg_pid = pid;
        int status;
        waitpid(pid, &status, 0);
        fg_pid = 0;
        printf("[wsh] foreground job %d finished\n", pid);
    }
    return 0;
}
```

mahij@Mahija: ~ × +

mahij@Mahija:~\$ gcc skill21.c -o skill21
mahij@Mahija:~\$./skill21
[wsh] signal handlers registered for SIGINT and SIGTSTP
hello
[wsh] foreground job 454 finished
mahij@Mahija:~\$ |

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ valgrind --leak-check=full --show-leak-kinds=all ./skill4
==838== Memcheck, a memory error detector
==838== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==838== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==838== Command: ./skill4
==838==
wsh> [buffer grown to 8 bytes]
stored: ls -la
wsh> stored: pwd
wsh> [buffer grown to 8 bytes]
stored: exit
-- history (most recent first) --
    exit
    pwd
    ls -la
==838==
==838== HEAP SUMMARY:
==838==      in use at exit: 0 bytes in 0 blocks
==838==    total heap usage: 13 allocs, 13 frees, 8,284 bytes allocated
==838==
==838== All heap blocks were freed -- no leaks are possible
==838==
==838== For lists of detected and suppressed errors, rerun with: -s
==838== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
mahij@Mahija:~$ |
```

 mahij@Mahija: ~ × +

```
mahij@Mahija:~$ cat process.c | grep -v exit | grep -v printf | tr a-z A-Z | sort | uniq | wc -l
15
[7-stage pipeline complete]
--exit code: 0
mahij@Mahija:~$ |
```

Skill 24

```
mahij@Mahija: ~ × +

mahij@Mahija:~$ cat README.md
# 2520030261_practical-skill-oss
Operating Systems & Systems Programming - incremental C shell project.
## Build
    gcc skillN.c -o skillN
## Structure
    src/      - one .c file per skill (skill2.c ... skill24.c)
    out/      - captured terminal output for each run
mahij@Mahija:~$ git add -A && git commit -m "Final review: organize source, add README, tag release"
mahij@Mahija:~$ git tag v1.0-final
mahij@Mahija:~$ git log --oneline
5e39b00 Final review: organize source, add README, tag release
mahij@Mahija:~$ git tag
v1.0-final
mahij@Mahija:~$ |
```